Mount Kenya University

Unlocking Infinite Possibilities

NAME: KAGOCHI BRIAN KAHORA

REG NO: BSCCS/2024/54776

COURSE: BACHELOR OF SCIENCE IN COMPUTER SCIENCE

SCHOOL: COMPUTING AND INFORMATICS

DEPARTMENT: INFORMATION TECHNOLOGY

UNIT CODE: BIT 2118

TITLE: SCIENTIFIC COMPUTING

ACADEMIC YEAR: SEP-DEC 2026

What the tasks showed

- **Speed.** Vectorised NumPy code ran about 16 to 105 times faster than a Python loop, and reusing an LU factorisation was about 23 times faster than solving each system from scratch.
- **Accuracy.** Floating-point numbers carry small rounding errors. Compare them with a tolerance, use Decimal for money, and remember that algebraically equal formulas can behave very differently.
- **Choosing a method.** Gauss-Seidel needed about half the iterations of Jacobi. Newton-Raphson found an IRR in 4 iterations where bisection needed 40, but bisection is the safer choice when you can bracket the root.
- **Models and data.** A cubic spline beat linear interpolation on a curved sensor response (mean error 0.132 vs 0.679 kPa), while a higher-degree curve fit can overfit a small dataset.
- **Simulation.** Smaller steps make Euler's method more accurate, solve_ivp adapts its own step size, and Monte Carlo estimates settle towards the true value as the number of samples grows.
- **Judgement.** A solver's answer is only as good as the assumptions behind it. Several real-world cases showed that results still need a sensible check, such as picking the physically meaningful root or adding practical constraints to an optimisation.

Repository and how to run the work

Repository: https://github.com/kahosh-dev/scientific-computing

The repository has three folders. The code folder holds the original Python programs along with my modified versions. The figures folder keeps the graphs those programs generate, and the docs folder has a write-up for each task covering the inputs, the operations, the outputs, the parameter experiments, how I read the graphs, and the modified application. Each task below has matching files named Task_NN in all three folders.

To run the work, first install the packages listed in requirements.txt. You can then run any single script, for example python code/Task_02_numpy_arrays.py, or run ./run_all.sh to go through the whole collection on a system that supports the script.

Task 2: NumPy Arrays and Numerical Computation

I wanted to see how a NumPy array can hold a run of measurements and let me calculate on all of them at once. An array stores values of one data type in a compact block, so functions like mean(),

min() and max() summarise the data quickly, and vectorised arithmetic reaches every element without a hand-written loop.

What I did. The program stores 24 hourly temperatures in °C, works out the daily mean, minimum and maximum, then converts the whole array to Fahrenheit using F = 1.8C + 32.

Main results

Measure	Result
Mean temperature	23.12 °C
Minimum	17.50 °C
Maximum	29.40 °C
First six Fahrenheit values	65.3, 64.4, 64.04, 63.5, 64.22, 66.2 °F

Changing a parameter. Adding 5 °C to every reading lifts the mean to 28.12 °C, the minimum to 22.50 °C and the maximum to 34.40 °C (93.92 °F). The curve keeps its shape because every value moves up by the same amount.

Real-world case. For my own version I analysed 24 hourly soil-moisture readings from a farm, ranging from 26% to 41%. The mean is 34%, and seven readings fall below an assumed 30% irrigation threshold. The program also calculates a wetness index between 0 and 1.

Takeaway. Arrays make it easy to summarise sensor data and spot when a field might need watering. The 30% threshold is only an assumption for the exercise; a real irrigation decision would depend on the crop, the soil and how well the sensor is calibrated.

Task 3: Mathematical Functions and Vectorization

This task puts an ordinary Python loop up against a NumPy vectorised operation, to see why vectorisation matters for large datasets. Vectorising means writing a calculation as one operation on a whole array, so NumPy does the element-by-element work in optimised compiled code and skips the overhead of a Python loop. I timed both versions with timeit.

What I did. I generated one million simulated latency readings between 5 and 250 milliseconds, converted each to seconds and squared it, once with a for loop and once with a single NumPy expression.

Main results

Measure	Result
Loop time	0.3623 s
Vectorised time	0.0221 s
Approximate speed-up	16.4 times

Changing a parameter. I varied the number of values from 1,000 up to 1,000,000. The vectorised version stayed faster throughout, with speed-ups of roughly 31 to 105 times. These ratios depend on memory, caching and the machine, so they will differ elsewhere.

Real-world case. My modified case works out the power dissipated by a 220-ohm load from one million simulated ADC readings (0–1023, 5 V reference), using $P = V^2/R$. The mean power is 37.91 mW and the peak is 113.64 mW. The loop took 1.05 s against 0.019 s for the vectorised version, about 55 times faster.

Takeaway. Both approaches give the same numbers, but vectorising is far quicker on large sensor datasets. The speed-up I measured depends on the hardware, so it shouldn't be treated as a fixed constant.

Task 4: Floating-Point Arithmetic and Numerical Errors

Here I wanted to understand why computer arithmetic can differ slightly from exact maths, and how that can make a calculation unreliable. Most decimal fractions can't be stored exactly in binary floating point, and the tiny rounding errors can build up. Catastrophic cancellation is the worst case: subtracting two nearly equal numbers wipes out many significant digits. Absolute error is the size of the gap from the true value, and relative error compares that gap with the true value.

What I did. The program tests 0.1 + 0.2 against 0.3, approximates √2 with 1.414, and compares $(\sqrt{(1+x)} - 1)/x$ with the algebraically equivalent but more stable form $1/(\sqrt{(1+x)} + 1)$.

Main results

Measure	Result
0.1 + 0.2	0.30000000000000004; direct equality with 0.3 is False; np.isclose is True
Approximation to √2	Absolute error 0.00021356; relative error 0.0151%
At $x = 10^{-12}$	Direct expression 0.500044; stable expression 0.500000

Changing a parameter. As x shrinks from 10^{-2} to 10^{-15}, the relative error of the direct expression climbs from about 1.5×10^{-14} to 0.11, while the stable expression stays close to 0.5.

Real-world case. My modified example is a bank-fee calculation that adds 0.01 one million times. The true total is 10,000.00. A float64 loop gives 10000.0000001719, whereas math.fsum and Decimal both give exactly 10000. Summing 100,000 fees in float32 gives about 1000.665 instead of 1000.

Takeaway. Floating-point values should be compared with a tolerance, for example np.isclose. For money, Decimal or whole cents are safer. The task also showed me that two formulas that are algebraically identical can behave very differently in practice.

Task 5: Matrix Algebra and Matrix Operations

This task uses matrices to represent geometric transformations and combine them. A rotation changes orientation, scaling changes size, and multiplying matrices chains transformations together.

The determinant gives the factor by which area changes, and its sign shows whether orientation has been flipped.

What I did. The original program transforms the four corners of a unit square. It combines a 45-degree rotation with scale factors of 2 horizontally and 1.5 vertically, then applies the combined matrix to each point.

Main results

Measure	Result
Transformed corners	(0, 0), (1.414, 1.414), (0.354, 2.475), (−1.061, 1.061)
Transformation determinant	3.0, so the area is three times the original

Changing a parameter. With the angle set to 90 degrees and the scale to (1, 3), the square becomes a 3-by-1 rectangle. The determinant is still 3, so the area scale factor hasn't changed.

Real-world case. In the modified case, a triangular game sprite is rotated 30 degrees about its centroid (2, 1) and then mirrored about the y-axis. The new vertices are (3.232, −0.866), (−0.232, 1.134) and (3.0, 2.732), and the area stays at 6.0.

Takeaway. A rotation has determinant +1 and a reflection has −1. The absolute value of the determinant tells you how area scales, and a negative sign means the shape has been flipped.

Task 6: Solution of Systems of Linear Equations

The aim was to solve simultaneous linear equations with matrix methods and read the answer in a practical setting. A system can be written as Ax = b, where A holds the coefficients, x the unknowns and b the known totals, and NumPy's solve() finds x when the solution is unique.

What I did. The coefficient matrix describes the CPU and memory used by two types of server. The totals are 20 CPU units and 72 GB, and np.linalg.solve(A, b) returns how many servers of each type are needed.

Main results

Measure	Result
Type A servers	3
Type B servers	4
Verification	A multiplied by the solution returns [20, 72]; residual is zero

Changing a parameter. If I change the totals to 24 CPU units and 80 GB, the answer becomes four servers of each type. The coefficient matrix stays the same; only the right-hand side changes.

Real-world case. For the modified example I applied Kirchhoff's voltage law to a circuit with three loops. The currents come out as I1 = 0.5015 A, I2 = 0.1522 A and I3 = −0.0896 A, with a residual norm of about 1.8×10^{-15} and a condition number of 3.8.

Takeaway. A negative current simply flows opposite to the direction I assumed. The tiny residual shows that the computed currents satisfy the equations very closely.

Task 7: Gaussian Elimination and LU Decomposition

This task looks at how a matrix can be factorised once and then reused to solve several systems efficiently. LU decomposition writes a matrix as a lower-triangular and an upper-triangular factor, usually with row pivoting for stability. Once the factors exist, different right-hand sides can be solved without repeating the costly factorisation.

What I did. I factorised a 3×3 matrix with SciPy's lu_factor, then reused the factors with lu_solve for three different measurement vectors, checking the residuals each time.

Main results

Measure	Result
Set 1	[5, 5, 5]
Set 2	[5.317, 5.268, 4.756]
Set 3	[4.902, 5.610, 5.537]
Residuals	Approximately 10^{-15} or zero

Changing a parameter. With a 400×400 matrix and 500 right-hand sides, solving each system from scratch took 0.732 s, while factorising once and reusing the factors took 0.031 s, about 23 times faster.

Real-world case. The modified case is a chain of four masses joined by springs, each with spring constant 200 N/m, written as Ku = f. A 10 N load on the last mass gives displacements of 50, 100, 150 and 200 mm. A 20 N load on the second mass gives 100, 200, 200 and 200 mm.

Takeaway. LU decomposition pays off when the same coefficient matrix is used with many load or measurement cases, because the factors can be reused instead of recomputed.

Task 8: Iterative Methods for Linear Systems

Here I compare two iterative ways of approximating the solution of a linear system. Both start from an initial guess and keep updating the unknowns. Jacobi uses values from the previous iteration, while Gauss-Seidel uses new values as soon as they are computed. A tolerance decides when to stop.

What I did. A 5×5 tridiagonal matrix models steady-state heat distribution along a rod, with boundary temperatures of 100 °C and 20 °C. Both methods stop once the largest change falls below 10^{-8}.

Main results

Measure	Result
Temperature solution	86.67, 73.33, 60.00, 46.67, 33.33 °C
Jacobi iterations	150
Gauss-Seidel iterations	75

Changing a parameter. With a tolerance of 10^{-3}, Jacobi needs 70 iterations and Gauss-Seidel 35. Raising the right-hand boundary to 50 °C changes the profile to 91.67, 83.33, 75.00, 66.67 and 58.33 °C.

Real-world case. I also solved a uniform metal rod with 19 interior points and end temperatures of 80 °C and 30 °C. Jacobi needed 1161 iterations and Gauss-Seidel 579, and the largest difference from the exact solution is about 4×10^{-5} °C.

Takeaway. In these examples Gauss-Seidel converges in about half as many iterations. A tighter tolerance gives better accuracy but usually costs more iterations, and larger systems take longer to converge.

Task 9: Eigenvalues and Eigenvectors

This task uses eigenvalues and eigenvectors to study a network's structure and the relative importance of its nodes. For a matrix A, an eigenvector v and its eigenvalue λ satisfy $Av = \lambda v$. In an adjacency matrix, the dominant eigenvector works as an importance score that reflects both how many connections a node has and how important its neighbours are.

What I did. A 4×4 adjacency matrix represents an undirected network of nodes A, B, C and D. NumPy computes the eigenvalues and eigenvectors, and I take the eigenvector for the largest eigenvalue, make it positive and normalise it so the scores add up to one.

Main results

Measure	Result
Dominant eigenvalue	2.5616
Scores	A 0.2192; B 0.2808; C 0.2808; D 0.2192
Eigenpair check	Norm of $Av - \lambda v$ is approximately 2×10^{-15}

Changing a parameter. Adding a link between A and D makes the network fully symmetric. The dominant eigenvalue becomes 3 and every node scores 0.25.

Real-world case. The modified case analyses a friendship network of six students with eight friendships. Cynthia scores highest (0.2242), followed by Dennis and Esther (0.1969 each), Felix (0.1386), and Amina and Brian (0.1217 each).

Takeaway. A node can score highly because it is linked to other well-connected nodes, not just because it has many direct links. The score captures a node's position in the network.

Task 10: Root Finding: Bisection and Fixed-Point Methods

The goal here is to find a numerical solution to an equation when an exact algebraic answer is awkward to get. A root is a value x where $f(x) = 0$. Bisection repeatedly halves an interval whose

endpoints have opposite signs. Fixed-point iteration rewrites the problem as x = g(x) and keeps evaluating g, and whether it converges depends on the function and the starting value.

What I did. The heat-equilibrium equation is f(T) = T − 25 − 10e^(−T/20). Bisection starts from a bracket containing a sign change, and fixed-point iteration uses T = 25 + 10e^(−T/20) starting at 30 °C.

Main results

Measure	Result
Bisection root	27.525211 °C in 39 iterations
Fixed-point root	27.525211 °C in 13 iterations

Changing a parameter. When I increase the heat-source coefficient from 10 to 30, the equilibrium rises to 31.279175 °C. Fixed-point iteration then needs 22 iterations instead of 13, because the iteration is less strongly contractive.

Real-world case. In the modified case, a loan of 5,000 is repaid in 24 monthly payments of 235. The estimated monthly interest rate is 0.986883%, which is a nominal annual rate of 11.843% and an effective annual rate of 12.507%. Total repayment is 5,640, of which 640 is interest.

Takeaway. Bisection is dependable as long as you know a bracket with a sign change, though it can be slow. Fixed-point iteration can be faster, but it needs a suitable rearrangement and the right convergence conditions.

Task 11: Root Finding: Newton-Raphson and Secant Methods

This task compares root-finding methods that use derivatives or successive function values. Newton-Raphson updates its estimate using the function and its derivative. The secant method estimates the derivative from the two most recent points, so no explicit derivative is needed. Both can converge quickly, but they rely on good starting estimates.

What I did. The program calculates the internal rate of return (IRR) for cash flows of −10,000 at the start, followed by 3,000, 3,500, 4,000 and 2,500 over four years. The IRR is the rate at which the net present value is zero.

Main results

Measure	Result
IRR	11.542461%
Newton-Raphson	4 iterations
Secant	7 iterations
Bisection (comparison)	40 iterations

Changing a parameter. When I raise the final cash inflow from 2,500 to 4,500, the IRR rises to 17.0937%. I also ran Newton-Raphson from a poorer starting guess of 50%.

Real-world case. The modified case is a bond with a face value of 1,000, an annual coupon of 6%, five years to maturity and a market price of 950. Its yield to maturity is 7.22687%, while the simple current yield is 6.316%.

Takeaway. A bond bought below face value can yield more than its coupon rate, because the investor also gains when it is redeemed at face value. Newton and secant are efficient when the starting points are sensible.

Task 12: Polynomial and Nonlinear Equation Solving

Here I solve systems of nonlinear equations numerically and think about what to do when there is more than one solution. scipy.optimize.root looks for values that make every residual equation zero, but different starting guesses can lead to different roots, or to no convergence at all.

What I did. The program finds where the circle $x^2 + y^2 = 4$ meets the parabola $y = x^2 - 1$. It calls scipy.optimize.root from several starting points and checks the residuals.

Main results

Measure	Result
Positive intersection	(1.5175, 1.3028)
Negative intersection	(−1.5175, 1.3028)
Residuals	Approximately 10^{-13}; solver reports success

Changing a parameter. Reducing the circle's radius to 1.5 moves the intersections to roughly (±1.3133, 0.7247). A starting guess of (0, 0) fails because the Jacobian is singular there.

Real-world case. In the modified case, a ball's path $y = x - 0.0109x^2$ is compared with a hill modelled by $(x - 60)^2 + y^2 = 30^2$. The intersections are (40.263, 22.593) and (89.948, 1.760), and the first is the physically relevant first impact.

Takeaway. A solver may return several mathematical roots, so it is up to the user to decide which one makes sense in the real situation, and to check the convergence status and residuals.

Task 13: Interpolation

This task estimates values between known measurements. Linear interpolation joins neighbouring points with straight lines, while cubic splines use smooth piecewise cubic polynomials and can follow curved trends. Both only estimate values inside the range of the known data.

What I did. I interpolated seven hourly temperatures from 08:00 to 14:00, using scipy.interpolate.interp1d and CubicSpline to give estimates every ten minutes.

Main results

Measure	Result
At 10:30, linear	23.25 °C
At 10:30, cubic spline	23.36 °C

Changing a parameter. I removed the 11:00 measurement and predicted it instead. Linear interpolation gives 23.75 °C and the cubic spline 24.11 °C, against a known value of 24.0 °C.

Real-world case. The modified case is a pressure-sensor calibration table that maps voltage to pressure. The two methods agree closely at some points and differ at others (see the table below), and a leave-one-out check shows the spline's mean error is far smaller.

Measure	Linear	Cubic spline
Estimate at 1.25 V	19.25 kPa	19.09 kPa
Estimate at 2.75 V	65.75 kPa	65.78 kPa
Leave-one-out mean error	0.679 kPa	0.132 kPa

Takeaway. The spline does better on this curved sensor response. Interpolation should stay within the measured voltage range of 0.5–4.5 V, because anything outside it is extrapolation and may be unreliable.

Task 14: Curve Fitting and Approximation

The aim is to fit a mathematical model to observed data and use it to describe trends or make predictions. Curve fitting finds the parameters that minimise the gap between measured and predicted values. The residual is the difference between an observation and the fitted value, and RMSE summarises the typical prediction error in the units of the data.

What I did. I fitted seven pairs of concurrent users and server response times with a quadratic model $y = ax^2 + bx + c$, using scipy.optimize.curve_fit.

Main results

Measure	Result
Fitted model	$y = 0.002167x^2 - 0.1345x + 85.00$
RMSE	1.55 ms
Prediction at 400 users	377.86 ms

Changing a parameter. Changing the model degree changes the fit error sharply. A more complex model matches the given points better but can overfit a small dataset.

Model degree	RMSE
Linear	18.83 ms
Quadratic	1.55 ms
Cubic	0.14 ms

Real-world case. For the modified case I fitted ten weeks of daily active users for a campus application with y = Ae^(kt). The model is 97.4e^(0.2199t), which means about 24.6% weekly

growth and a doubling time of 3.15 weeks. RMSE is 3.8 users, R^2 is 0.9998, and the week-12 forecast is about 1,364 users.

Takeaway. A good fit to past data doesn't guarantee accurate predictions, and exponential growth can't go on forever when the number of potential users is limited.

Task 15: Numerical Differentiation

This task estimates derivatives from discrete measurements and uses them to interpret rates of change. Forward and backward differences use points on one side, while central differences use points on both sides and are often more accurate.

What I did. GPS positions in metres are recorded from t = 0 to 6 seconds. The program estimates speed with a forward difference at the first point, central differences at the interior points and a backward difference at the last point.

Main results

Measure	Result
Estimated speed range	4.80 m/s at t = 0 to 8.00 m/s at t = 6
Example	5.65 m/s at t = 2 s

Changing a parameter. For sin(x) at x = 1, the central-difference error shrinks roughly with the square of the step size, while the forward-difference error shrinks roughly in proportion to it. Once the step gets extremely small, round-off error makes the results worse again.

Real-world case. In the modified case, a car's speed is logged every two seconds as it accelerates from 0 to 78 km/h, and np.gradient estimates its acceleration. Acceleration peaks at 1.94 m/s² at 4 seconds, drops to 0.07 m/s² at 20 seconds, and averages 1.08 m/s².

Takeaway. The car accelerates hardest at the start and then settles towards a nearly constant speed. The step size has to balance truncation error against floating-point round-off.

Task 16: Numerical Integration

This task estimates accumulated quantities, such as energy or volume, from measurements taken over time. Integration can be read as the area under a curve. The trapezoidal rule joins data points with straight segments, while Simpson's rule fits parabolic segments and is often more accurate for smooth functions.

What I did. Smart-meter power readings in kilowatts are recorded every two hours over twelve hours. Integrating power over time gives an estimate of energy use in kilowatt-hours.

Main results

Measure	Result
Trapezoidal estimate	15.700 kWh
Simpson estimate	15.800 kWh
Difference	0.100 kWh

Changing a parameter. For sin(x) from 0 to π the exact integral is 2. Using more points reduces both errors, and Simpson's rule improves faster on this smooth function.

Real-world case. In the modified case, water inflow is measured every 15 minutes for two hours. The estimated volume is 3157.5 L by the trapezoidal rule and 3175.0 L by Simpson's rule, with an average flow of 26.46 L/min. A 3000 L tank would fill within that period.

Takeaway. Numerical integration turns a changing rate into a total amount. The two water-volume estimates are close, so either method would do here, as long as the measurements are good.

Task 17: Ordinary Differential Equations: Euler Method

Here I approximate the solution of a differential equation step by step with Euler's method. An ordinary differential equation links a quantity to its rate of change, and Euler uses the current slope to estimate the next value: $y(n+1) = y(n) + h \cdot f(tn, yn)$, where h is the step size.

What I did. I modelled Newton's law of cooling, $dT/dt = -k(T - Tambient)$, with k = 0.12 per minute, an ambient temperature of 25 °C and an initial temperature of 90 °C. Euler's method takes one-minute steps for 30 minutes.

Main results

Measure	Result
Euler temperature after 30 min	26.40 °C
Exact temperature	26.78 °C
Absolute error	0.372 °C

Changing a parameter. Reducing the step from 5 to 0.1 minutes cuts the final error from 1.5098 °C to 0.0383 °C. Very large steps can make the numerical solution oscillate or become unstable.

Real-world case. In the modified case, a fish population follows logistic growth from an initial 50 fish, with a growth rate of 0.4 per month and a carrying capacity of 1000. The table shows the estimates at 24 months against the exact value.

Step size	Euler estimate at 24 months
h = 4	1003.06
h = 1	999.53
h = 0.25	998.94
Exact	998.71

Takeaway. Smaller steps generally make Euler more accurate but need more calculations. The logistic model shows growth slowing as the population nears the environment's carrying capacity.

Task 18: Numerical Solution of ODEs Using SciPy

This task uses a scientific library to solve differential equations with adaptive numerical methods. SciPy's solve_ivp handles initial-value problems and adjusts its step size to meet the error tolerances I set, which suits equations where a fixed, simple step could be wasteful.

What I did. I modelled an RC charging circuit with dV/dt = (Vs − V)/(RC), where Vs = 5 V, R = 1000 ohms, C = 0.001 F and the initial voltage is zero. The solution is computed over five seconds and compared with the exact exponential formula.

Main results

Measure	Result
Solver status	Success
Capacitor voltage at 5 s	4.9663 V
Maximum error against exact curve	9.3×10^{-10} V

Changing a parameter. Doubling the resistance to 2000 ohms doubles the time constant from one second to two. The voltage at one second falls from about 3.161 V to 1.967 V, and at five seconds from 4.966 V to 4.590 V.

Real-world case. The modified case uses the SIR epidemic model for susceptible, infected and recovered students in a school of 1000. Reducing the contact rate β flattens and delays the infection peak.

Setting	Infection peak	Total infected
β = 0.5, γ = 0.2	235 on day 19	about 893
β = 0.3, γ = 0.2	66 on day 40	about 588

Takeaway. The RC example shows how component values affect charging speed. The SIR example shows that less contact flattens and delays the peak, although the model still predicts an outbreak while the reproduction number stays above one.

Task 19: Numerical Optimization

This task finds values of decision variables that minimise or maximise an objective while meeting constraints. The feasible region is the set of values that satisfy every constraint, and a numerical optimiser searches it for a low-cost or high-benefit solution.

What I did. The program minimises the cost 12x + 18y for two server types, subject to capacity 100x + 170y being at least 1000 and x and y being non-negative. I used SciPy's SLSQP optimiser, followed by a search over whole numbers.

Main results

Measure	Result
Continuous optimum	x = 0, y = 5.882
Minimum continuous cost	105.88
Best whole-number choice	0 Type A and 6 Type B; capacity 1020; cost 108

Changing a parameter. When the cost of Type B rises from 18 to 30, the preferred solution switches to 10 Type A and no Type B, at a total cost of 120.

Real-world case. In the modified case, a canteen meal is built from portions of rice, beans and vegetables, minimising cost while providing at least 25 g protein, 600 kcal and 10 g fibre. Under the exercise assumptions, the solution is 1.765 portions of beans, costing 0.882, with 37.1 g protein, 600 kcal and 26.5 g fibre.

Takeaway. The best mathematical answer depends on the objective and the constraints. A beans-only meal isn't realistic or balanced, which is why practical constraints such as variety and maximum portions need to be included.

Task 20: Random Numbers and Monte Carlo Methods

This task uses repeated random simulation to estimate probabilities and get a feel for uncertainty. Monte Carlo methods generate many random samples and estimate a result from the outcomes. The estimate usually becomes more stable as the number of simulations grows, but it always carries some sampling uncertainty.

What I did. Server demand is modelled as a normal random variable with a mean of 850 and a standard deviation of 120, and the server capacity is 1000. I generate one hundred thousand simulated demands and take the fraction above capacity as the overload probability.

Main results

Measure	Result
Simulated overload probability	10.616% (10,616 out of 100,000)
Exact normal probability	10.565%

Changing a parameter. Changing the capacity changes the overload risk sharply, as the table shows. Increasing the number of simulations from 100 to 1,000,000 makes the estimate settle closer to the theoretical value. A separate simulation with 200,000 points estimates π as 3.146.

Server capacity	Estimated overload probability
900	33.6%
1000	10.6%
1100	1.9%
1200	0.17%

Real-world case. In the modified case, I simulated the duration of a project with three sequential tasks (design, build and testing) using triangular duration distributions. The mean duration is 27.33 days with a standard deviation of 3.69 days. The probability of finishing within 30 days is 76.28%, and the 90th percentile is 32.3 days.

Takeaway. Monte Carlo simulation helps estimate risks that are hard to calculate directly. The project example shows that adding up only the most likely task durations can underestimate the time needed; a deadline with 90% confidence is about 32 days.

Source

Kahosh-dev, Scientific Computing repository: https://github.com/kahosh-dev/scientific-computing. Task write-ups are in the docs folder, implementations in the code folder and generated plots in the figures folder.